

Goal:

Find the indices of two numbers in an array (input by user) that add up to a target number.

Approach:

Instead of the hefty brute force approach or the sorting approach which would cause a huge toll on the time complexity, we decide to trade off with the space complexity and maintain its balance with the time complexity. This allows our implementation to run without taking up too much time while still maintaining an acceptable space tradeoff. The approach I find suitable for this solution is to use a hash map.

Step 1:

We create a dictionary called, lets say “met”, that stores all the input elements as the traversal begins.

Defined in line 28 of the `task1.py` file as:

```
met = {}
```

Step 2:

We then use a for loop defined as

```
for i in range(len(user)):
```

This line helps loop the array provided by the user with the help of the index `i`

Step 3:

As the loop starts, the complement of the target with the integer element at the current index is calculated with

```
complement = target - user[i]
```

Step 4:

The loop peeks at the `met` dictionary and verifies if the element at the current index `i` is in the dictionary. If it exists, our solution is found. If it isn't, the loop continues and stores the element and its index using

```
met[user[i]] = i
```